Name: . . . . . Number: . . . . . .

CSC211 Data Structures and Algorithms

More on growth functions

Term 1 Tutorial 2 Thursday 19 February 2026

Programs can be classified in terms of their running time, usually stated in terms of the size their input n. For, example if one sorts n numbers, the timing of the sorting is given in terms of this n and not in terms of the actual instructions that are executed to do the sorting. Timing may be expressed as $T(n)$, the total time taken, e.g., $T(n) = \frac{1}{7}2^n + 75n^2 + 119n + 23$. When the size of the input n gets very big, the value of $T(n)$ becomes so dominated by the 2^n term, that nothing else really matters. Then, without defining it clearly, Sedgewick expresses the value of $T(n)$ *assymptotically* as $T(n) \sim O(n^2)$. Knuth in 1973, and Aho, Hopcroft and Ullman (AHU) in 1974, and Cormen, Leiserson, Rivest and Stein (CLRS) in 2022 write $T(n) = O(n^2)$.

Sometimes we can easily see how to express the timing $T(n)$ in terms of $T(n)$, e.g., $T(n) = 2T(n) + cn$. We can show that this is $O(n \log(n)$

Fill in the gaps

1. For the towers of Hanoi problem it will take
   (a) $T(1) =$ 1 move to move *one disk* from the **from** tower to the **target** tower.
   (b) $T(2) =$ 3 moves to move *two disks* from the **from** tower to the **target** tower.
   (c) $T(3) =$ 7 moves to move *three disks* from the **from** tower to the **target** tower.
   (d) Moving *five disks* will take $T(5) =$ 31 moves.
   (e) Moving n *disks* will take $T(n) =$ $2^n - 1$ moves.
   (f) How can $T(N)$ be expressed in terms of $T(N-1)$?
   $T(N) = 2T(N-1) + 1$

2. What is the asymptotic complexity of the following program? $O($ n^2 $)$

```
public class Q2 {
    public static void main(String[] args) {
        int n = 5;

        // Two nested loops T(n) = ....
        for (int i = 0; i < Math.log(n); i++) {
            for (int j = 0; j < n; j++) {
                System.out.print("*␣"); }
            System.out.println(); } } }
```

3. What is the asymptotic complexity of the following program? $O(\log(n))$

```
public class Q3 {
    public static void main(String[] args) {
        int n = 2048;

        for (int i = n; i > 0; i /= 2) { // Outer loop .... iterations:

            for (int j = 0; j < 16; j++) { // Inner loop: constant time
                System.out.print(i + "␣"); }
            System.out.println(); } } }
```

4. What is the asymptotic complexity of the following program? . . . $O(\log(n))$

```java
public class Q4 {
    public static void main(String[] args) {
        int n = 16;

        for (int i = 1; i < n; i *= 2) { // Outer loop .... iterations:
            for (int j = 0; j < 16; j++) { // Inner loop: constant time
                System.out.print(i + "␣"); }
            System.out.println(); } } }
```

5. What is the asymptotic complexity of the following program? . . . $O(n\log(n))$

```java
public class Q5 {
    public static void main(String[] args) {
        int n = 16;

        for (int i = 0; i < n; i++) { // Outer loop: .... iterations
            for (int j = n; j > 0; j /= 2) {// Inner loop: .... iterations
                System.out.print("*␣"); }
            System.out.println(); } } }
```

6. What is the asymptotic complexity of the following program? . . . .

```java
public class Q6 {
    public static void main(String[] args) {
        int n = 16;
        generateSubsets("", n); }

    static void generateSubsets(String prefix, int n) {
        if (n == 0) {
            System.out.println(prefix);
            return; }

        generateSubsets(prefix + "0", n − 1);
        generateSubsets(prefix + "1", n − 1); } }
```

$O(1)$

$T(n) = 2T(n-1) + 1$

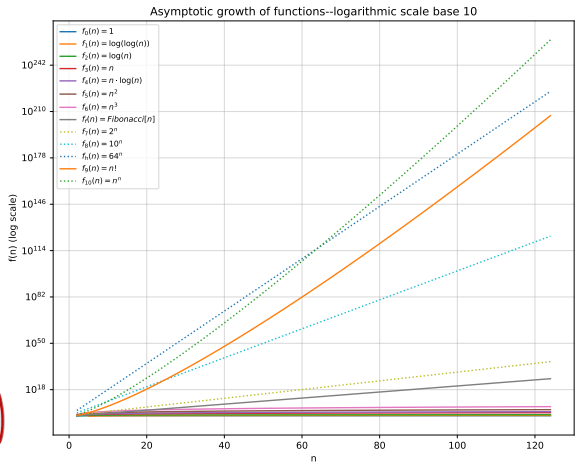
$T(n-1) = 2T(n-2) + 1$

$T(n-2) = 2T(n-3) + 1$

$T(n) = 4T(n-2) + 3$

$= 8T(n-3) + 7$

$T_n = 2^K T(n-K) + (2^K - 1)$



$n - k = 0 \Rightarrow n = K$

$T(n) = 2^n(T(0) + (2^n - 1))$

$= 2^n(1) + 2^n - 1 = 2^{n+1} - 1 \Rightarrow T(n) \sim O(2^n)$

(Remove constants)